

# GenON 공고와 연결한 Agent/RAG 문제 해결 경험

GenON 공고를 **고객사별 업무·데이터를 테스트 가능한 Agent PoC로 바꾸고**, GenOS 테스트페이지에서 **지표·로그로 검증하는 역할**로 읽었습니다. 3개 프로젝트를 **테스트페이지·고객사 커스터마이징·모델 학습 데이터셋 관리·Agent 생성** 관점으로 재정리했습니다.

## APPLICATION MOTIVATION

저는 GenON의 업무를 모델을 붙이는 데서 끝내지 않고, 고객사별 실패 원인을 테스트페이지·API·로그·데이터셋으로 확인 가능하게 만드는 일로 보았습니다. PyMate는 RAG 품질, Workflow Agent는 Agent 신뢰성, Seoul Culture Map은 고객사 PoC의 비용·라우팅 문제를 보여주는 근거입니다.

## LINKS



## PORTFOLIO STRUCTURE

# 프로젝트 구성 의도

GenON 업무를 **테스트 가능한 Agent PoC**를 만들고, **고객사 데이터에 맞게 검증·커스터마이징하는 일**로 해석했습니다. 그래서 3개 프로젝트를 아래 흐름으로 재배열했습니다.

### 01 · PYMATE

#### RAG 품질 진단

부정확한 답변을 **검색 병목**과 평가 지표로 분해했습니다.

### 02 · WORKFLOW AGENT

#### Agent 신뢰성 검증

고신뢰 오답을 **Guardrail·데이터셋** 문제로 다뤘습니다.

### 03 · SEOUL CULTURE MAP

#### 고객 PoC 비용/라우팅

질의 유형별 경로를 나눠 **비용·지연**을 줄였습니다.

### GENON FIT

#### 테스트·커스터마이징

실패 원인을 로그와 지표로 확인하는 **Agent 테스트 구조**로 연결했습니다.

읽는 기준은 하나입니다. **기능을 만들었다**가 아니라, 문제를 어떻게 정의했고 어떤 구조로 검증했으며 그 결과가 GenON의 고객사 Agent 업무에 어떻게 쓰이는지 보이도록 구성했습니다.

37→85%

#### Agent 정확도

Guardrail + confidence  
보정

0.83→0.97

#### RAG Precision

Embedding + hybrid  
retrieval

70→97%

#### JSON Validity

Structured output  
validation

85%↓

#### LLM Cost

Intent routing으로 호출 절감

## JD FIT SUMMARY

# 공고에서 읽은 역할과 제 경험의 연결

GenON 공고를 **테스트 가능한 Agent 흐름**을 만들고, **고객사별 실패 원인을 지표·로그·데이터셋으로 좁히는 역할**로 읽었습니다. 아래 4개 관점만 먼저 제시하고, 세부 근거는 각 프로젝트의 GenON 적용 포인트에서 풀었습니다.

### 테스트 가능한 Agent

FastAPI·SSE·로그·평가 지표로 Agent 응답 실패를 화면과 기록에서 확인할 수 있게 만들었습니다.

### 고객사 커스터마이징

문서 검색, 판단 Agent, Guardrail, 비용 라우팅을 분리해 도메인별 교체가 가능한 구조로 설계했습니다.

### 데이터셋/평가 루프

RAGAS와 오답 로그 19건 재구성으로 품질 하락 원인을 데이터 단위에서 다시 추적했습니다.

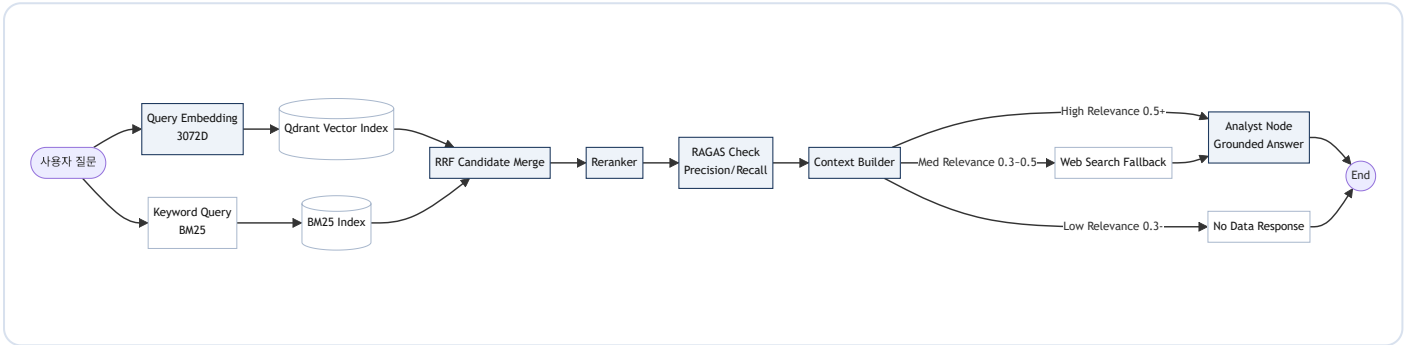
### API/운영화

REST endpoint, SSE streaming, provider abstraction, Docker/AWS 배포로 실험을 서비스 형태로 연결했습니다.

“답변이 부정확하다”는 피드백을 **LLM 문제**로 단정하지 않고, RAGAS로 검색/생성 병목을 분해해 **임베딩·검색 파이프라인 문제**로 재정의한 프로젝트입니다.

|                                      |                                                                         |                                              |                                                  |                                                   |
|--------------------------------------|-------------------------------------------------------------------------|----------------------------------------------|--------------------------------------------------|---------------------------------------------------|
| <b>일정</b><br>2026.01 ~ 2026.02<br>6주 | <b>기술 스택</b><br>Python, Django, Qdrant, RAGAS, BM25, HyDE, RRF, AWS EC2 | <b>참여 인력</b><br>팀 4명<br><b>RAG 평가·임베딩 담당</b> | <b>서비스</b><br>Python 공식 문서·강의 자료 기반 학습 보조 RAG 챗봇 | <b>인프라 기준</b><br>반복 질의 사용을 가정해 검색 품질과 응답 지연 균형 우선 |
|--------------------------------------|-------------------------------------------------------------------------|----------------------------------------------|--------------------------------------------------|---------------------------------------------------|

ARCHITECTURE · RAG EVALUATION & RETRIEVAL FLOW



|                                                                                                               |                                                                                                   |                                                                                                  |
|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>요청 흐름</b><br>질문을 <b>semantic/vector</b> 와 <b>keyword/BM25</b> 두 경로로 보내 후보를 넓힌 뒤 RRF와 reranker로 근거를 압축했습니다. | <b>핵심 판단</b><br>LLM 교체보다 먼저 RAGAS로 검색 병목을 분리했고, Context Precision <b>0.83</b> 을 구조 변경 기준으로 삼았습니다. | <b>검증 지점</b><br>relevance threshold에 따라 답변, 웹 보강, no-data 응답을 나누어 테스트페이지에서 실패 원인을 추적할 수 있게 했습니다. |
|---------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|

문제 원인

- 사용자 피드백은 “답변이 부정확”이었지만, 원인이 **LLM 생성**인지 **검색 후보 회수**인지 분리되지 않았습니다.
- 강의 자료와 Python 공식 문서의 용어 차이 때문에 초기 벡터 검색만으로는 필요한 근거를 충분히 회수하지 못했습니다.
- 모델 교체부터 진행하면 비용은 늘지만, 실제 병목이 retrieval이면 품질 개선이 제한될 수 있었습니다.

해결 과정

- “부정확한 답변”을 검색 실패, 근거 부족, 생성 불일치로 나누고 RAGAS 4지표로 병목을 측정했습니다.
- Context Precision **0.83**을 검색 병목으로 보고, LLM 교체보다 **embedding 768D→3072D** 전환을 먼저 결정했습니다.
- HyDE+BM25+RRF로 후보 회수를 넓히고, reranker는 마지막 단계에만 배치해 품질·지연 trade-off를 조정했습니다.
- Flask MVP를 Django/EC2 배포 구조로 옮겨 반복 테스트 가능한 서비스 형태로 정리했습니다.

결과

- Context Precision **0.83→0.97**, Context Recall **0.70→0.79**로 검색 품질을 개선했습니다.
- “LLM 교체 전 검색 품질부터 확인한다”는 판단 기준을 만들었습니다.
- GenON 고객사 문서 RAG의 **품질 진단·커스터마이징·테스트페이지 지표**로 확장 가능한 경험입니다.

GenON 적용 포인트

고객사 문서 기반 RAG 솔루션을 만들 때 “LLM을 바꾸면 해결된다”는 접근보다 **검색 후보 회수·재랭킹·평가 지표**를 테스트페이지에서 확인할 수 있게 만드는 것이 중요합니다. 이 경험은 GenOS 테스트페이지의 품질 점검 항목과 고객사별 RAG 커스터마이징에 바로 연결됩니다.

Retrospective

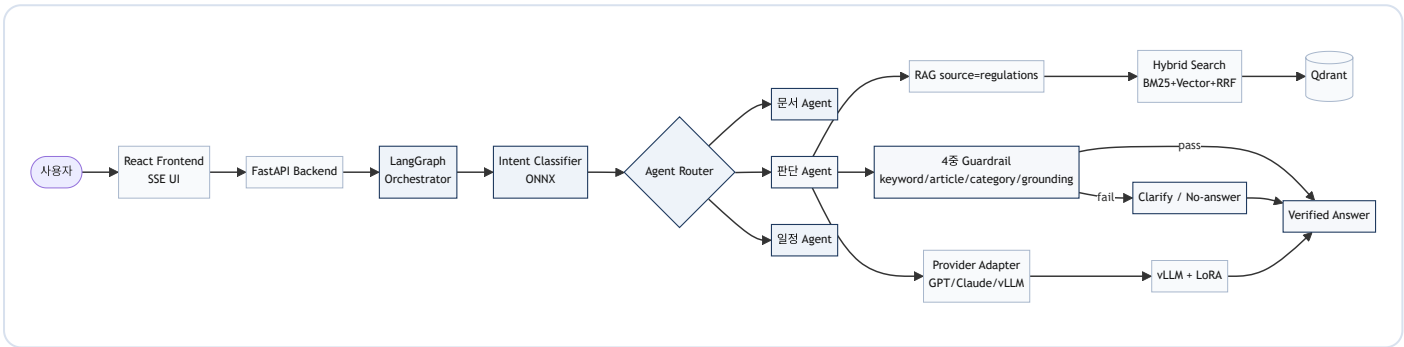
이 프로젝트에서 얻은 가장 큰 기준은 “답변 품질 문제를 바로 모델 문제로 해석하지 않는다”였습니다. 검색 후보 회수, 재랭킹, 평가 지표를 먼저 분리해 보면 고객사 문서 RAG에서도 개선 우선순위를 더 빠르게 좁힐 수 있습니다.

# WorkFlow Agent

LLM Agent가 “모른다”고 말해야 할 상황에서 높은 confidence로 오답을 내는 문제를 **서버 검증 게이트와 데이터셋 재구성**으로 해결한 사내 규정 업무 보조 Agent 프로젝트입니다.

|                                      |                                                                                   |                                               |                                                           |                                                            |
|--------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------|
| <b>일정</b><br>2026.02 ~ 2026.04<br>8주 | <b>기술 스택</b><br>LangGraph, FastAPI, vLLM,<br>LoRA, Qdrant, Docker,<br>RunPod A100 | <b>참여 인력</b><br>팀 4명<br><b>검증·서빙 파이프라인 담당</b> | <b>서비스</b><br>사내 규정 질의에 근거와 판단<br>결과를 제공하는 업무 보조<br>Agent | <b>운영 기준</b><br>오답이 업무 의사결정에 영향을<br>주므로 정확도보다 검증 가능성<br>우선 |
|--------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------|-----------------------------------------------------------|------------------------------------------------------------|

## ARCHITECTURE · AGENT RELIABILITY GATE



|                                                                                            |                                                                                                        |                                                                                    |
|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>요청 흐름</b><br>React SSE UI에서 FastAPI로 들어온 요청을 LangGraph가 의도 분류 후 문서·판단·일정 Agent로 라우팅합니다. | <b>핵심 판단</b><br>판단 Agent는 RAG 근거와 모델 응답을 분리해 생성하고, provider adapter로 GPT/Claude/vLLM 교체 실험이 가능하게 했습니다. | <b>검증 지점</b><br>서버의 4중 Guardrail이 조항 존재, 카테고리, 근거 일관성, 출력 형식을 확인해 고신뢰 오답 노출을 막습니다. |
|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|

### 문제 원인

- Base model은 존재하지 않는 규정에도 confidence **0.92**를 출력했고, 단순 threshold로는 고신뢰 오답을 막기 어려웠습니다.
- 근거 없는 답변, 카테고리 오류, JSON 형식 오류가 한 덩어리로 섞여 있어 실패 유형별 조치가 어려웠습니다.
- LoRA v2에서 데이터 98건을 추가했지만 정확도가 **-3.2%p** 하락해, 데이터 양보다 품질 문제가 더 크다는 신호가 있었습니다.
- 고객 업무 Agent에서는 환각이 기능 오류가 아니라 비즈니스 리스크로 이어지므로 모델 바깥 검증 레이어가 필요했습니다.

### 해결 과정

- 환각, 근거 없음, 카테고리 오류, JSON 실패를 먼저 분리해 실패 유형별 검증 기준을 정의했습니다.
- 키워드 매칭, 조항 존재 검증, 카테고리 제한, 답변-근거 일관성 검사의 **4중 Guardrail**을 설계했습니다.
- LLM confidence를 그대로 쓰지 않고 검색 점수, 근거 존재, 출력 형식, 카테고리 일치를 결합해 보정했습니다.
- 오답 로그 **19건**만 선별해 LoRA v3 데이터셋을 재구성하고, GPT/Claude/vLLM을 provider 패턴으로 묶었습니다.

### 결과

- 규정 판단 정확도 **37%→85%**, JSON 유효율 **70%→97%**로 개선했습니다.
- 고신뢰 오답 confidence를 **0.92→0.78**로 보정해 서비스 노출 전 검증 기준을 만들었습니다.
- GenON의 **Agent 생성·고객사 규정 문서 커스터마이징·데이터셋 관리** 업무와 직접 연결됩니다.

### GenON 적용 포인트

GenON의 고객사 Agent 생성 업무에서 가장 중요한 것은 “Agent가 답한다”가 아니라 **Agent가 언제 답하지 말아야 하는지**를 정하는 것입니다. 이 프로젝트는 고객사 규정·업무 문서에 맞춘 Guardrail, 데이터셋 생성, 테스트페이지 검증 시나리오로 확장할 수 있습니다.

### Retrospective

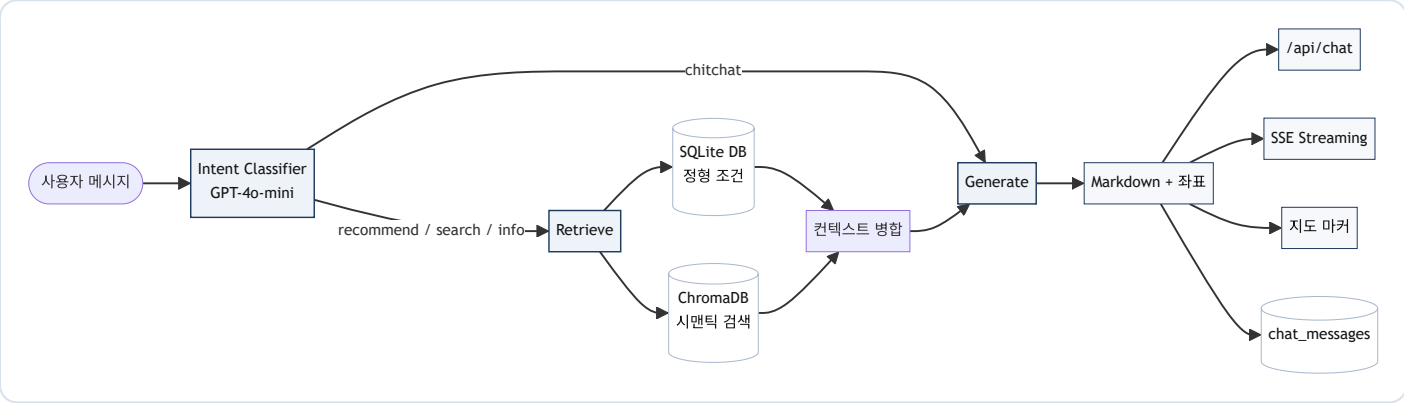
모델 정확도를 높이는 실험보다 더 중요했던 것은 실패 케이스를 어떤 기준으로 라벨링할지였습니다. v2 실패 이후 “데이터 양”이 아니라 “실패 유형을 겨냥한 데이터셋”이 Agent 신뢰성 개선에 더 강하게 작동한다는 기준을 얻었습니다.

# Seoul Culture Map

서울시 문화시설 2,500+개 데이터를 기반으로 자연어 질의를 처리하면서, 모든 요청을 LLM에 태우지 않고 **Intent Routing**으로 비용과 응답 지연을 줄인 Agent PoC입니다.

|                                             |                                                                                    |                                          |                                            |                                                        |
|---------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------|--------------------------------------------|--------------------------------------------------------|
| <b>일정</b><br>2026.03 ~ 진행중<br><b>개인 PoC</b> | <b>기술 스택</b><br>FastAPI, LangGraph,<br>OpenAI API, ChromaDB,<br>SQLite, SSE, React | <b>참여 인력</b><br>개인<br><b>문제 정의~배포 전담</b> | <b>서비스</b><br>서울시 문화시설 조건 검색·<br>추천형 Agent | <b>트래픽 기준</b><br>월 1만 요청 가정, 호출 비용과<br>첫 응답 시간 중심으로 설계 |
|---------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------|--------------------------------------------|--------------------------------------------------------|

## ARCHITECTURE · INTENT ROUTING & HYBRID RETRIEVAL



|                                                                                                       |                                                                                                 |                                                                                |
|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>요청 흐름</b><br><b>/api/chat</b> 요청을 Intent Classifier가 먼저 분류하고, 필요한 경우에만 SQLite/ChromaDB 검색 경로를 탑니다. | <b>핵심 판단</b><br>정형 조건은 SQL filter, 의미 기반 추천은 vector search로 분기해 "모든 요청을 LLM에 태우는" 비용 구조를 피했습니다. | <b>검증 지점</b><br>SSE 첫 응답 시간, 요청당 비용, 지도 마커/좌표 출력 여부를 테스트페이지에서 확인할 수 있게 설계했습니다. |
|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|

### 문제 원인

- 단순 인사, 조건 검색, 추천 질문이 같은 LLM 경로를 타면 불필요한 호출 비용이 커지고 응답 시간이 늘어났습니다.
- 서울시 문화시설 **2,500+**개 데이터는 정형 필터와 의미 검색이 섞여 있어 단일 검색 방식으로는 사용자 의도를 놓치기 쉬웠습니다.
- 고객사 PoC에서는 기능 동작뿐 아니라 월 요청량 기준 비용과 응답 지연을 함께 설명해야 했습니다.

### 해결 과정

- 일상 대화, 조건 검색, 추천 질의를 먼저 나누고 Intent→Retrieve→Generate **3-node** 구조로 분리했습니다.
- LLM 호출 전 검색 필요 여부를 먼저 판단하고, 정형 조건은 SQLite/SQL filter로 처리했습니다.
- 의미 기반 추천 요청은 ChromaDB vector search로 라우팅하고, 두 검색 결과를 컨텍스트로 병합했습니다.
- SSE streaming으로 첫 토큰을 빠르게 보여주고, 월 1만 요청 기준 API 비용을 비교했습니다.

### 결과

- 요청당 비용 **\$0.020→\$0.003**, 월 1만 요청 환산 **~\$200→~\$30**로 약 85% 절감했습니다.
- 첫 토큰 약 **500ms**, 응답 완료 평균 약 **2.4s**로 테스트 가능한 Agent PoC를 만들었습니다.
- GenON 고객사별 트래픽/도메인에 따라 **Agent 경로와 테스트 시나리오**를 조정하는 역량을 보여줍니다.

### GenON 적용 포인트

고객사마다 트래픽 분포와 질의 유형이 다르기 때문에, Agent PoC에서는 모델 성능뿐 아니라 **어떤 요청을 어떤 경로로 보낼지**가 비용과 안정성을 좌우합니다. 이 경험은 GenOS 테스트페이지에서 시나리오별 비용·응답 시간을 보여주는 방식으로 활용할 수 있습니다.

### Retrospective

개인 PoC를 만들며 Agent 구조는 모델 선택보다 요청 유형을 어떻게 나누는지가 먼저라는 기준을 얻었습니다. 고객사별 데이터와 트래픽이 달라질수록 Intent Routing, 비용 추정, 테스트페이지 로고를 함께 설계해야 운영 가능한 PoC가 됩니다.